

Analysis of Algorithms

Kasun Karunanayaka
Senior Lecturer, UCSC

This presentation is made using the following sources

<http://www.cs.iupui.edu/~cs240/summer10/slides/t17CAlgorithmAnalysisBigOh.ppt>

<https://medium.com/karuna-sehgal/a-simplified-explanation-of-the-big-o-notation-82523585e835>

<https://www.chegg.com/tutors/what-is-Asymptotic-and-Unbounded-Behavior/>

<https://medium.com/karuna-sehgal/a-simplified-interpretation-of-the-time-complexity-and-space-complexity-78e4cdbb0b19>

Time Complexity and Space Complexity

- ☛ Designing algorithms is about making decisions and decisions have tradeoffs.
 - In the shortest time
 - Using the least amount of space
- ☛ So, the complexity can be broken down into
 - Time Complexity -> How long does the algorithm take
 - Space Complexity -> How much space does the algorithm use



Real world examples

- ☞ An example of an algorithm that may be more time efficient: Time Efficient Grocery Shopping
 - Drive a large car to the grocery store.
 - Buy everything that you need.
 - So, this algorithm saves time but requires you to have a large car. This means lower time complexity and higher space complexity
- ☞ An example of an algorithm that may be more space efficient: Space Efficient Grocery Shopping
 - Walk to the grocery store with a tote bag.
 - Buy only one item.
 - Walk home and drop off the item.
 - Repeat until done.
 - This algorithm takes a long time but does not require a car. This means higher time complexity and lower space complexity.

Algorithms

- ◆ Computer Scientists focus on problems such as
 - How fast do algorithms run
 - How much memory does the process require
- ◆ Example Applications
 - Make the Internet run faster
 - Pink-Degemark's routing algorithms
 - Gene Meyers determined the sequences of the Human genome using his whole genome shotgun algorithm

Analysis of Algorithms

- ◆ We have ways to compare algorithms
 - Generally, the larger the problem, the longer it takes the algorithm to complete
 - Sorting 100,000 elements can take much more time than sorting 1,000 elements
 - and more than 10 times longer
 - the variable n suggests the "*number of things*"
 - If an algorithm requires $0.025n^2 + 0.012n + 0.0005$ seconds, just plug in a value for n

A Computational Model

- ◆ To summarize algorithm runtimes, we can use a computer independent model
 - instructions are executed sequentially
 - count all assignments, comparisons, and increments there is infinite memory
 - every simple instruction takes one unit of time

Simple Instructions

◆ Count the simple instructions

- assignments have cost of 1
- comparisons have a cost of 1
- let's count all parts of the loop
 - `for (int j = 0; j < n; j++)`
 - `j=0` has a cost of 1, `j<n` executes $n+1$ times, and `j++` executes n times for a total cost of $2n+2$
- each statement in the repeated part of a loop have have a cost equal to number of iterations

Examples

```
sum = 0;  
sum = sum + next;
```

Cost
-> 1
-> 1 *Total Cost: 2*

```
for (int i = 1; i <= n; i++)  
    sum = sum++;
```

Cost
-> 1 + n+1 + n = 2n+2
-> n *Total Cost: 3n + 2*

```
k = 0  
for (int i = 0; i < n; i++)  
    for (int j = 0; j < n; j++)  
        k++;
```

Cost
-> 1
-> 2n+2
-> n(2n+2) = 2n² + 2n
-> n² *Total Cost: 3n² + 4n + 3`*

Total Cost of Sequential Search

Cost

<code>for (int index = 0; index < n; index++)</code>	->	$2n + 2$
<code> if(searchID.equals(names[index]))</code>	->	n
<code> return index;</code>	->	$0 \text{ or } 1$
 <code>return -1 // if not found</code>	->	$0 \text{ or } 1$

Total cost = $3n+3$

Different Cases

- ◆ The total cost of sequential search is $3n + 3$
 - But is it always exactly $3n + 3$ instructions?
 - The last assignment does not always execute
 - But does one assignment really matter?
 - How many times will the loop actually execute?
 - that depends
 - If searchID is found at index 0: _____ iterations
 - best case
 - If searchID is found at index $n-1$: _____ iterations
 - worst case

Typical Case of sequential (linear)

- ◆ The average describes the more typical case
- ◆ First, let the the entire cost be simplified to n
 - Assume the target has a 50/50 chance of being in the array
 - n comparisons are made: worst-case occurs 1/2 the time
 - Assume if it's in **a**, it's as likely to be in one index as another

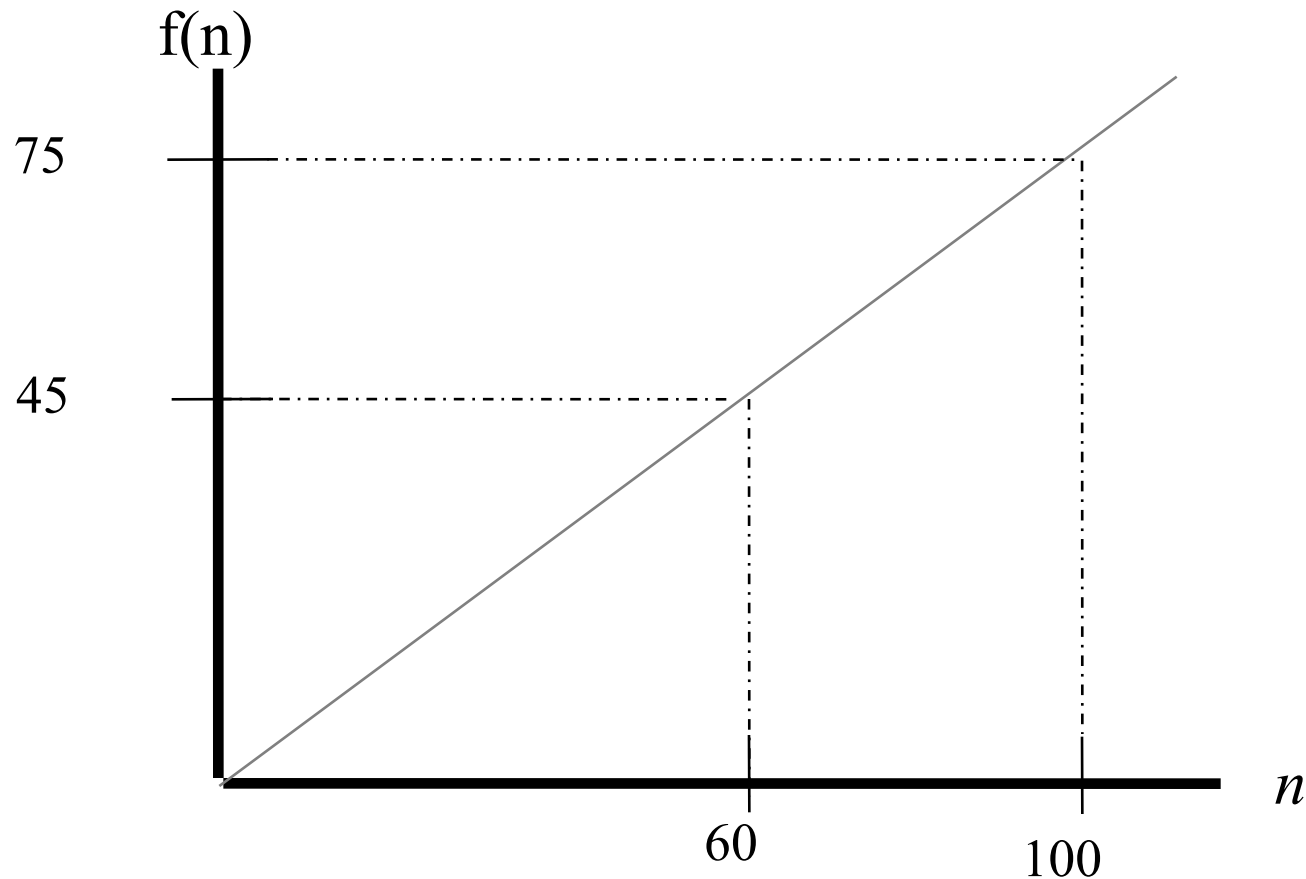
$$\frac{1}{2}n + \frac{1}{2} \times \frac{n}{2} = \frac{n}{2} + \frac{n}{4} = \frac{3}{4}n$$

Half the time it is n comparisons, the other half it is $n/2$ comparisons

- So the typical case is $3/4 n$ comparisons

The Essence of Linear Search

Plot the function *this is why sequential search is also called linear search. As n increases, runtime forms a line*



Linear Search Continued

- ◆ This equation is a polynomial: $3n + 3$
- ◆ The fastest growing term is the high-order term
- ◆ The high order term (which could be n^2 or n^3), represents the most important part of the analysis function
- ◆ We refer to the rate of growth as the *order of magnitude*, which measure the rate of growth

Rate of Growth

♦ Imagine two functions:

$$f(n) = 100n \qquad g(n) = n^2 + n$$

- When n is small, which is the bigger function?
- When n is big, which is the bigger function?
- We can say: *$g(n)$ grows faster than $f(n)$*

Rate of Growth, another view

Function growth and weight of terms as a percentage of all terms as n increases for

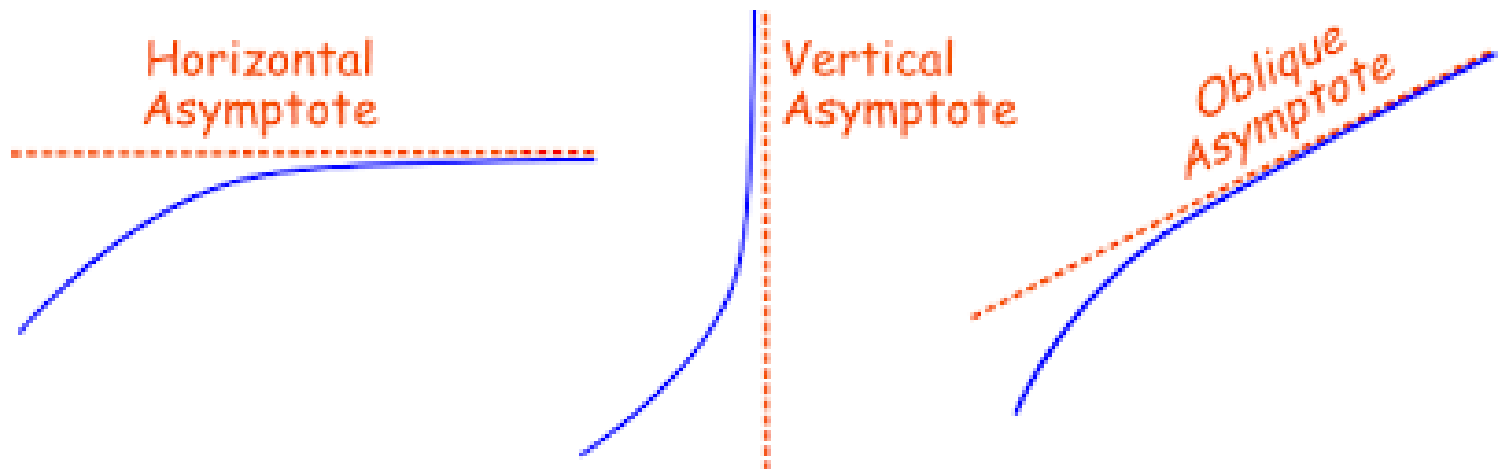
$$f(n) = n^2 + 80n + 500$$

n	f(n)	n^2	80n	500
10	1,400	100 (7%)	800 (57%)	500 (36%)
100	18,500	10,000 (54%)	8,000 (43%)	500 (3%)
1000	1,0805,000	1,000,000 (93%)	80,000 (7%)	500 (0%)
10000	100,800,500	100,000,000 (99%)	800,000 (1%)	500 (0%)

Conclusion: consider highest order term with the coefficient dropped, also drop all lower order terms

What is Asymptotic and Unbounded Behavior?

- Asymptote: a straight line that continually approaches a given curve but does not meet it at any finite distance.
- Asymptotic behavior** describes a function or expression with a defined limit or asymptote. Your function may approach this limit, getting closer and closer as you change the function's input, but will never reach it.
- Unbounded behavior** is when you have a function or expression without any limits. As you increase your function's input, the output also increases



Asymptotic and Unbounded Behavior - Examples

- ☞ Example for Asymptotic expression is division, such as $1/x$. No matter how big x gets, you will never quite make it to having exactly nothing.
 - Cutting a piece of Pizza
- ☞ Unbounded behavior is when you increase your function's input, the output also increases. A good example of this is multiplication, or $1 \cdot x$.
 - If you have 5 ovens, you can cook 5 pizzas at once; if you had an infinite number of ovens, you could cook an infinite number of pizzas.

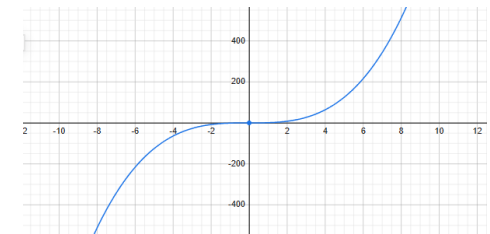
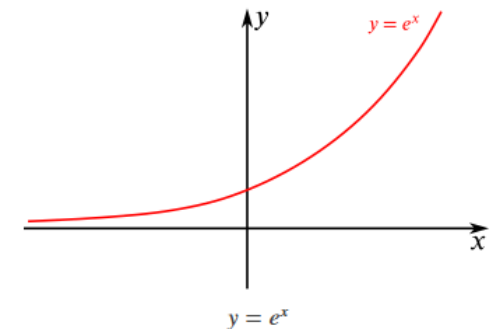
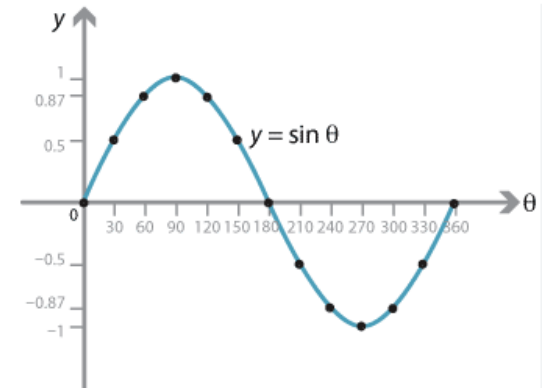
Asymptotic and Unbounded Behavior

☞ $f(x) = 1/x$

- ☞ Here, if you plug in infinity for x , you get 1 over a really huge number (infinity). Now, this doesn't mean that $1/\text{infinity}=0$, but we can say that there's an asymptote at 0, meaning the values of $f(x)$ get REALLY close to 0, but never actually touch 0. This is an asymptote.
- ☞ A function is unbounded when the limit of the function tends to infinity or negative infinity. A function that fluctuates is also unbounded.

Bounded and Unbounded Functions

- A very simple bounded function is **$y = \sin(x)$** . Its upper bound for example 1, because this function never has greater value than 1, and its lower bound for example -1 , because it never has smaller value than -1 .
- A simple unbounded function is **$y = e^x$** , because it has no upper bound, it increases to $+\infty$. Though it has a lower bound, for example 0.
- **$f(x) = x^3$** This function has neither an upper nor lower bound



Asymptotic and Unbounded Behavior

- ☛ Many functions have interesting behavior as their input grows to infinity, or as they themselves grow without a bound as they approach a certain value.
- ☛ While infinity, or numbers growing without end, seems hard to study, the concepts of limits and asymptotes enables to describe the behavior of functions as they approach the unreachable infinity.

What is the Big O Notation?

- Big O Notation is the language we use to describe the complexity of an algorithm.
- In other words, Big O Notation is the language we use for talking about how long an algorithm takes to run.
- It is how we compare the efficiency of different approaches to a problem.
- With Big O Notation, we express the runtime in terms of — how quickly it grows relative to the input, as the input gets larger.

What is the Big O Notation?

Let's break down the last line:

- ☛ **how quickly the runtime grows** — It is difficult to determine the exact runtime of an algorithm. It depends on the speed of the computer processor. So instead of talking about the runtime directly, we use Big O Notation to talk about how quickly the runtime grows.
- ☛ **relative to the input** — With Big O Notation, we use the size of the input, which we call “ n ”. So we can say things like the runtime grows “on the order of the size of the input” ($O(n)$) or “on the order of the square of the size of the input” ($O(n^2)$).
- ☛ **as the input gets larger** — Our algorithm may have steps that seem expensive when n is small but are eclipsed eventually by other steps as n gets larger. For Big O Notation analysis, we care more about the stuff that grows fastest as the input grows, because everything else is quickly eclipsed as n gets very large.

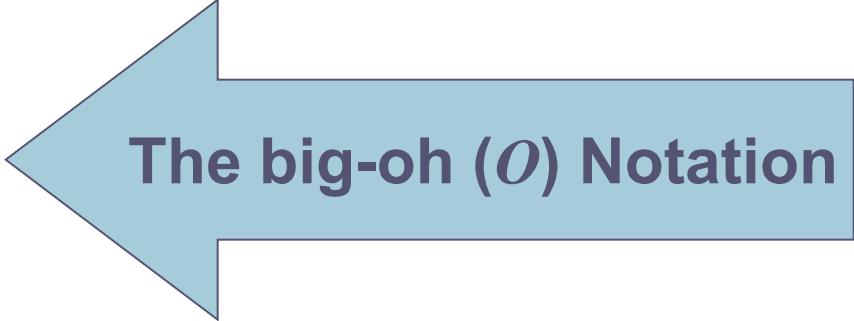
Asymptotic Analysis

- Ignoring constants in $T(n)$
- Analyzing $T(n)$ as n "gets large"

Example: $T(n) = 13n^3 + 42n^2 + 2n\log n + 4n$

As n grows larger, n^3 is MUCH larger than n^2 , $n\log n$, and n , so it dominates $T(n)$

The running time grows roughly on the order of
 $T(n) = O(n^3)$



The big-oh (O) Notation

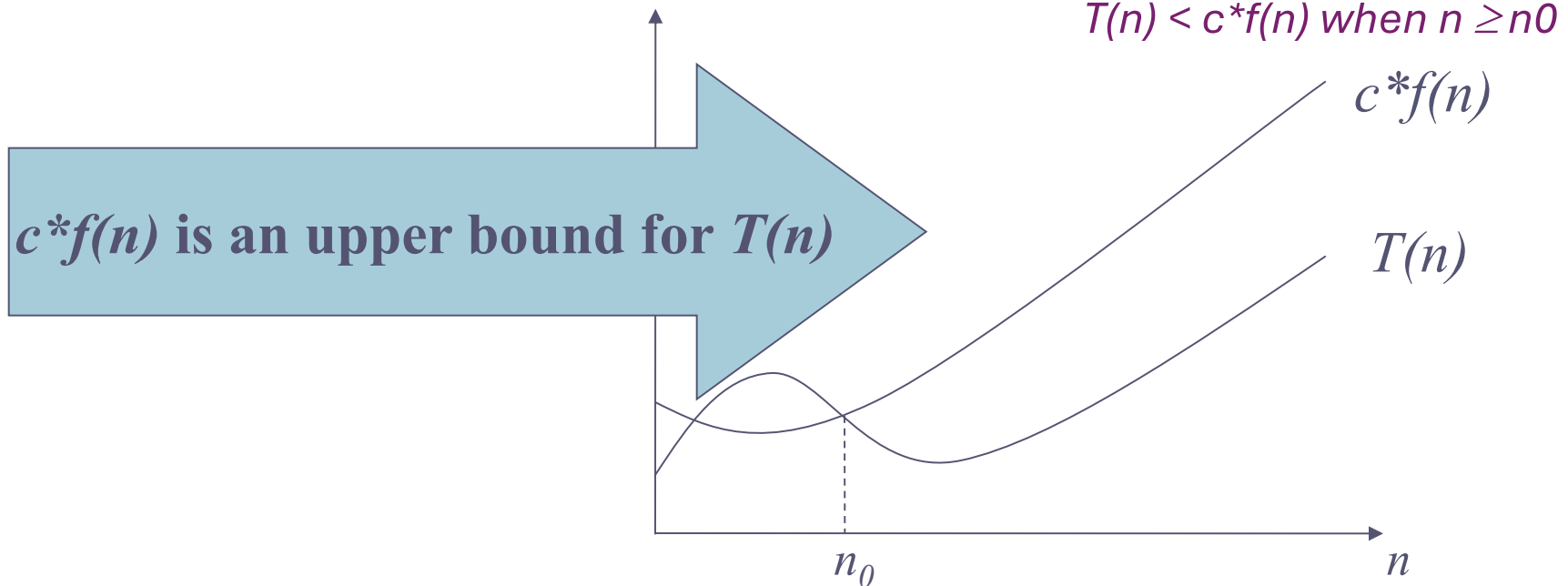
Big-Oh Defined

The O symbol was introduced in 1927 to indicate relative growth of two functions based on asymptotic behavior of the functions now used to classify functions and families of functions

The Big O notation defines an upper bound of an algorithm, it bounds a function only from above. For example, consider the case of Insertion Sort. It takes linear time in best case and quadratic time in worst case. We can safely say that the time complexity of Insertion sort is $O(n^2)$. Note that $O(n^2)$ also covers linear time.

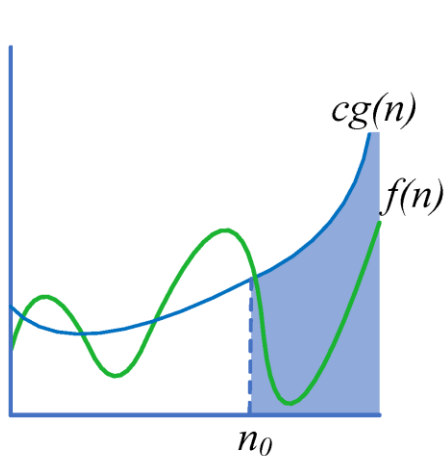
$T(n) = O(f(n))$ if there are constants c and n_0 such that

$$T(n) < c * f(n) \text{ when } n \geq n_0$$

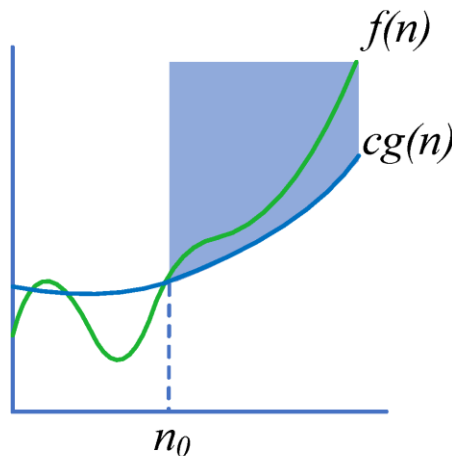


3 major notations

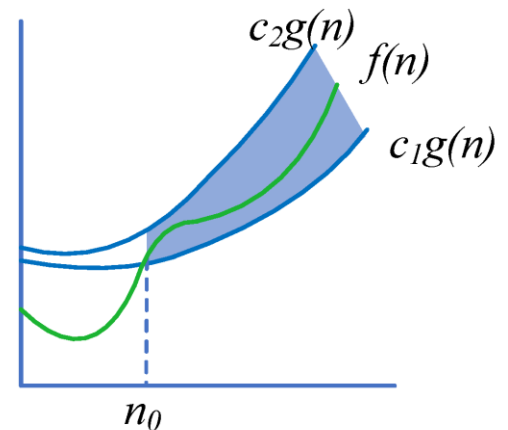
- ☛ $O(g(n))$, Big-Oh of g of n , the Asymptotic Upper Bound.
- ☛ $\Omega(g(n))$, Big-Omega of g of n , the Asymptotic Lower Bound.
- ☛ $\Theta(g(n))$, Big-Theta of g of n , the Asymptotic Tight Bound.



(a) Big-O Notation



(b) Big-Omega Notation



(c) Big-Theta Notation

Can we justify Big O notation?

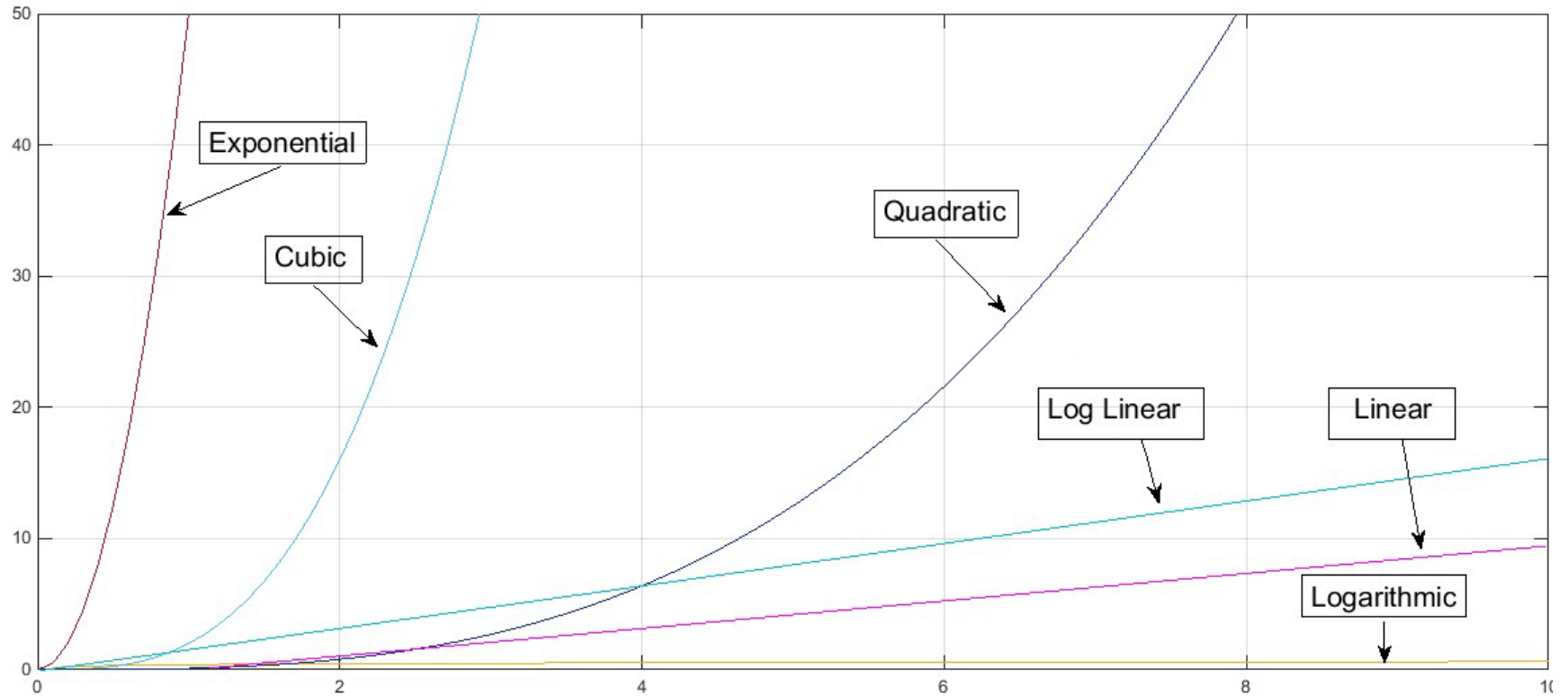
Big O notation is a *huge* simplification; can we justify it?

- It only makes sense for *large* problem sizes
- For sufficiently large problem sizes, the highest-order term swamps all the rest!

Consider $R = x^2 + 3x + 5$ as x varies:

$x = 0$	$x^2 = 0$	$3x = 10$	$5 = 5$	$R = 5$
$x = 10$	$x^2 = 100$	$3x = 30$	$5 = 5$	$R = 135$
$x = 100$	$x^2 = 10000$	$3x = 300$	$5 = 5$	$R = 10,305$
$x = 1000$	$x^2 = 1000000$	$3x = 3000$	$5 = 5$	$R = 1,003,005$
$x = 10,000$				$R = 100,030,005$
$x = 100,000$				$R = 10,000,300,005$

Different Big O Curves



Rules for Calculating Big-Oh

Base of Logs ignored

$$\log_a n = O(\log_b n)$$

Power inside logs ignored

$$\log(n^2) = O(\log n)$$

Base and powers in exponents not ignored

$$3^n \text{ is not } O(2^n)$$

$$a^{(n^2)} \text{ is not } O(a^n)$$

If $T(x)$ is a polynomial of degree n , then $T(x) = O(x^n)$

Some Examples of Big O Notation

- ☛ This function runs in $O(1)$ time (or “constant time”) relative to its input. This means that the input array could be 1 item or 1,000 items, but this function would still just require one “step.”

```
function printFirstItem(arrayOfItems) {  
    console.log(arrayOfItems[0]);  
}
```

Some Examples of Big O Notation

- ☛ This function runs in $O(n)$ time (or “linear time”), where n is the number of items in the array. This means that if the array has 10 items, I have to print 10 times. If it has 1,000 items, I have to print 1,000 times.

```
function printAllItems(arrayOfItems) {  
    arrayOfItems.forEach(function(item) {  
        console.log(item);  
    });  
}
```

Some Examples of Big O Notation

- Both of these functions have $O(n)$ runtime, even though one takes an integer as its input and the other takes an array:

```
function sayHelloNTimes(n) {  
    for (var i = 0; i < n; i++) {  
        console.log('hello');  
    }  
}  
  
function printAllItemsInArray(theArray) {  
    theArray.forEach(function(item) {  
        console.log(item);  
    });  
}
```

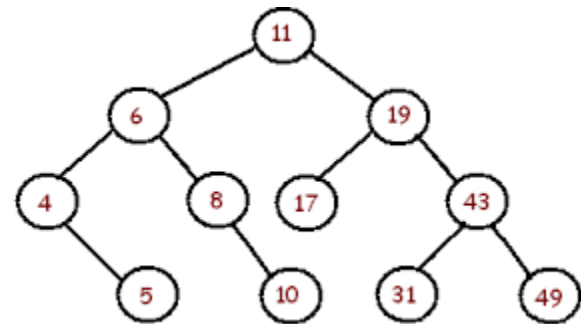
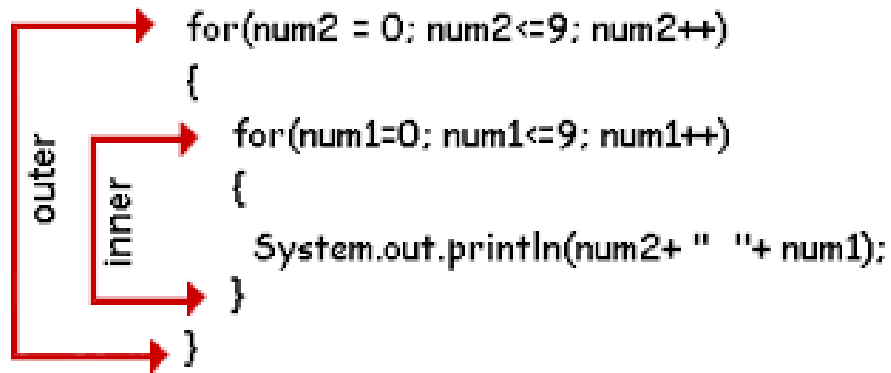
Some Examples of Big O Notation

- In this example, I might have 100 items in the haystack, but the first item might be the needle, in which case I would return in just 1 iteration of the loop. I can say this is $O(n)$ runtime and the worst case scenario would be implied. But to be more specific I could say this is worst case $O(n)$ and best case $O(1)$ runtime.

```
function contains(haystack, needle) {  
    for (var i = 0; i < haystack.length; i++) {  
        if (haystack[i] === needle) return true;  
    }  
    return false;  
}
```


Some Examples of Big O Notation

- When you have two nested for loops $O(n^2)$
- Binary tree search $O(\log n)$



Space complexity

- Space complexity refers to the amount of memory or storage space that an algorithm or function consumes based on the size of its input. It measures how the memory usage of an algorithm scales with the size of the input data.

Some Examples of Big O Notation - Space Complexity

- ☛ This function takes $O(1)$ space (not allocating any new variables)

```
function sayHelloNTimes(n) {  
    for (var i = 0; i < n; i++) {  
        console.log('hello');  
    }  
}
```

Some Examples of Big O Notation - Space Complexity

- ☛ This function takes $O(n)$ space (the size of helloArray scales with the size of the input):

```
function printFirstItem(arrayOfItems) {  
    console.log(arrayOfItems[0]);  
}
```

SELF STUDY

Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$	$O(\log(n))$
Mergesort	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Timsort	$O(n)$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Heapsort	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell Sort	$O(n)$	$O((n \log(n))^2)$	$O((n \log(n))^2)$	$O(1)$
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

Thank You!